

Term Project

Implementation of Python-style Vector Operations in a LISP Environment



202170116 윤민석

지도교수: 우균

제출일: 2025년 12월 20일

부산대학교

목차

1	서론	3
2	프로그래밍 언어 설계	4
2.1	어휘 구조 (Lexical Structure)	4
2.2	구문 구조 (Syntactic Structure)	5
2.3	의미론 (Operational Semantics)	5
2.3.1	평가(Evaluation) 및 할당	5
2.3.2	다형성 연산 (Polymorphism)	6
2.3.3	벡터 연산 (Vector Operations)	6
3	프로그래밍 언어 구현	8
3.1	구현 방향 및 시스템 개요	8
3.1.1	인터프리터 방식의 채택	8
3.1.2	시스템 구성도 (System Architecture)	8
3.2	구문 분석 및 AST 생성	8
3.2.1	어휘 분석과 토큰 처리 (Lexer)	8
3.2.2	일반화 리스트를 이용한 AST 구조	9
3.3	타입 시스템 및 메모리 구조	10
3.3.1	동적 타이핑과 Tagged Union	10
3.3.2	타입 검사 (Type Checking)	10
3.4	평가기(Evaluator) 및 연산자 구현	11
3.4.1	다형성 연산자 구현 (Polymorphism)	11
3.4.2	벡터 연산 및 정렬 (Vector Operations)	11
4	테스트 코드와 테스트 결과	13
4.1	테스트 환경 및 출력 형식	13
4.2	기초 연산 및 변수 처리 (Basic Operations)	13

4.3	문자열 다형성 연산 (Polymorphic String Ops)	14
4.4	벡터 생성 및 조회 (Vector Read Operations)	15
4.5	동적 벡터 수정 (Vector Modification)	16
4.6	고급 기능: 벡터 산술 및 재귀 정렬 (Advanced Features)	17
5	고찰 (Reflections)	19
5.1	타입 확장의 필요성과 설계적 선택	19
5.2	다형성 연산 구현 과정에서의 고민	20
5.3	벡터 자료구조 설계와 대안 비교	20
5.4	AST 구성 방식 선택에 대한 고찰	21
5.5	인터프리터 방식 선택의 타당성	21
5.6	전체 구현 과정에서 얻은 교훈	21
6	결론	22

제1장 서론

본 프로젝트는 기존 실습 과정에서 단계적으로 구축해 온 LISP 인터프리터의 기능적 한계를 보완하고, 이를 현대적 스크립트 언어에 가까운 형태로 확장하고자 하는 동기에서 출발하였다. 이전까지 구현된 인터프리터는 정수(Integer) 타입만을 변수에 저장할 수 있었으며, 자료구조 역시 cons 셀 기반의 연결 리스트에 국한되어 있었다. 이러한 구조는 LISP의 작동 원리를 학습하기에는 충분하지만, 실제 프로그래밍 언어가 갖추어야 할 데이터 표현력과 조작 능력 측면에서는 분명한 제약을 가지고 있었다. 이에 따라 정수뿐만 아니라 문자(Character)와 문자열(String)까지 포괄하는 보다 유연한 타입 체계를 도입하고, 사용자가 직관적으로 데이터를 다룰 수 있는 환경을 구축하는 것을 주요 목표로 설정하였다.

이를 위해 본 프로젝트에서는 Python 언어가 제공하는 단순하고 강력한 연산 모델에서 영감을 받아, 연산자 오버로딩을 통한 다형적 연산(Polymorphic Arithmetic)을 구현하였다. 덧셈(+)은 숫자 합산뿐 아니라 문자열 결합을 수행하고, 곱셈(*)은 숫자 연산뿐 아니라 문자열 반복을 지원하도록 설계함으로써, LISP 인터프리터 위에서 Python과 유사한 표현력을 제공한다. 또한 기존 cons 기반 리스트 구조의 한계를 보완하기 위해 벡터(Vector) 자료구조를 새롭게 도입하였다. 벡터는 임의 접근(Random Access)에 유리하고, 정수-문자-문자열-벡터 등 서로 다른 타입이 혼합된 이기종 데이터(Heterogeneous Data)를 하나의 컨테이너에 저장할 수 있도록 설계되었다. 이를 위해 내부적으로 Tagged Union 방식을 활용하여 제네릭 벡터(Generic Vector) 역할을 수행하는 자료구조를 구현하였다.

마지막으로, 이러한 벡터 자료구조의 활용성을 높이기 위해 연산 모듈을 별도로 분리하여 삽입·삭제·수정 등 기본적인 CRUD 기능뿐 아니라 선형 탐색, 슬라이싱, 스택 연산(push/pop), 그리고 중첩된 자료형까지 비교 가능한 재귀적 정렬(sorting) 기능을 제공하였다. 이와 같은 일련의 확장을 통해 본 프로젝트는 작은 규모의 인터프리터이지만 실제 프로그래밍 언어에 가까운 표현력과 실용성을 갖춘 LISP 환경을 구축하는 것을 목표로 한다.

제2장 프로그래밍 언어 설계

본 장에서는 프로젝트를 통해 설계한 확장된 LISP 언어의 구문론(Syntax)과 의미론(Semantics)을 정의한다. 구문론은 다시 어휘 구조(Lexical Structure)와 구문 구조(Syntactic Structure)로 나누어 기술하며, 의미론은 프로그램의 실행 방식과 연산의 동작 원리를 자연어로 기술한다. 특히 본 언어는 기존 LISP의 S-expression 구조를 유지하면서 벡터(Vector) 자료형과 다형성 연산을 지원하기 위해 확장된 문법을 채택하였다.

2.1 어휘 구조 (Lexical Structure)

어휘 구조는 프로그램의 입력 문자열을 의미 있는 토큰(Token)으로 분류하는 규칙을 정의한다. 본 언어의 어휘 구조는 정규 정의(Regular Definition)를 사용하여 다음과 같이 정의할 수 있다.

- 기본 정의

<i>digit</i>	→ [0-9]
<i>alpha</i>	→ [a-zA-Z]
<i>symbol</i>	→ [-+*/%]=]
<i>escape</i>	→ \n \t \r \\ \"

- 토큰 정의

<i>number</i>	→ <i>digit</i> ⁺
<i>identifier</i>	→ <i>alpha</i> (<i>alpha</i> <i>digit</i>) [*]
<i>operator</i>	→ <i>symbol</i>
<i>char</i>	→ ' <i>any_character</i> '
<i>string</i>	→ " (<i>escape</i> [^\"])* "

위 정의에서 *number*는 정수형 리터럴을, *identifier*는 변수명 및 함수명을 나타낸다. 특히 *string*은 큰따옴표 내부에 이스케이프 시퀀스(*escape*)를 허용하여 문자열 처리를 유연하게

설계하였다. 괄호 기호 (,), [,]는 구문 구조에서 구분자로 사용되므로 별도의 토큰 규칙으로 정의하지 않고 리터럴로 처리한다.

2.2 구문 구조 (Syntactic Structure)

구문 구조는 어휘 분석을 통해 생성된 토큰들이 결합되어 문장을 형성하는 규칙을 정의한다. 본 언어의 문법은 문맥 자유 문법(CFG, Context-Free Grammar)으로 기술하며, 벡터와 리스트를 구분하기 위한 생성 규칙을 포함한다. 구문 구조는 다음과 같다.

- 문법 규칙 (Grammar Rules)

```

Program  → {Line}
Line     → Expr \n | \n
Expr     → Atom
          | List
          | Vector
Atom     → number | identifier | operator | char | string
List     → ( {Expr} )
Vector   → [ {Expr} ]

```

여기서 *List*는 소괄호 ()를 사용하는 전통적인 LISP의 연결 리스트 구조를 나타내며, *Vector*는 대괄호 []를 사용하여 생성되는 동적 배열 구조를 의미한다. *Expr*은 재귀적으로 정의되어 리스트나 벡터 내부에 또 다른 리스트나 벡터가 중첩될 수 있음을 나타낸다.

2.3 의미론 (Operational Semantics)

언어의 의미론은 구문 구조로 표현된 문장이 실행될 때의 동작을 설명한다. 본 언어는 **적용적 순서(Applicative Order)** 평가 방식을 따르며, 주요 기능에 대한 수행 의미는 다음과 같다.

2.3.1 평가(Evaluation) 및 할당

- **심볼 평가**: *Expr*이 *identifier*인 경우, 현재 환경(Environment)에서 바인딩된 값을 반환한다. 정의되지 않은 심볼 참조 시 UNDEFINED를 출력한다.
- **벡터 평가 (Eager Evaluation)**: *Vector* 구문 [*e*₁ *e*₂ ...]가 평가될 때, 내부의 모든 요소 *e*_{*i*}가 즉시 평가된 후 그 결과값들을 요소로 갖는 새로운 벡터 객체가 생성된다.

- **할당 (Assignment):** ($= id Expr$)

$Expr$ 을 평가한 값을 id 에 바인딩한다. 이는 부수 효과(Side-effect)를 발생시키며 환경을 갱신한다.

2.3.2 다형성 연산 (Polymorphism)

본 언어의 산술 연산자는 피연산자의 타입에 따라 다형적으로 동작한다.

- **덧셈 연산:** ($+ A B$)

- A, B 가 정수/문자인 경우: 산술 덧셈을 수행한다. (문자는 ASCII 값으로 처리)
- A, B 가 문자열인 경우: 두 문자열을 연결(Concatenation)한다.
- A, B 가 벡터인 경우: 두 벡터를 병합하여 새로운 벡터를 생성한다.

- **곱셈 연산:** ($* A B$)

- A, B 가 정수/문자인 경우: 산술 곱셈을 수행한다.
- A 가 문자열/벡터이고 B 가 정수인 경우: A 를 B 회 반복(Repetition)한 새로운 객체를 생성한다.

2.3.3 벡터 연산 (Vector Operations)

벡터 자료형을 조작하기 위해 다음과 같은 내장 함수(Built-in Functions)들의 의미를 정의한다.

- ($vec\ e_1 \dots$): 인자들을 요소로 갖는 새로운 벡터를 생성한다.
- ($len\ v$): 벡터 v 의 길이를 반환한다.
- ($idx\ v\ i$): 벡터 v 의 i 번째 요소를 반환한다. (0-based index)
- ($slice\ v\ s\ e$): 벡터 v 의 구간 $[s, e)$ 을 잘라낸 새로운 벡터를 생성한다. 이 연산은 얇은 복사(shallow copy) 방식으로 이루어지며, 원본 벡터 내부 요소를 복제하지 않고 동일한 Data 포인터를 공유한다.
- ($set\ v\ i\ val$): 벡터 v 의 i 번째 요소를 val 로 덮어쓴다. 인덱스가 범위를 벗어날 경우 NIL을 반환한다.
- ($ins\ v\ i\ val$): 벡터 v 의 i 번째 위치에 val 을 삽입하고, 이후 요소들을 오른쪽으로 한 칸씩 이동시킨다.

- `(del v i)`: 벡터 v 의 i 번째 요소를 제거한 뒤, 이후 요소들을 왼쪽으로 이동시켜 연속성을 유지한다.
- `(find v val)`: 벡터 v 내부에서 val 을 선형 탐색하여 첫 번째로 일치하는 인덱스를 반환한다. 존재하지 않으면 `NIL`을 반환한다.
- `(push v val)` / `(pop v)`: 벡터 v 를 스택처럼 사용하여 요소를 뒤에 추가하거나 제거한다.
- `(sort v)`: 벡터 v 의 요소들을 오름차순으로 정렬한다. 요소 비교는 타입 계층($\text{Nat} < \text{Chr} < \text{Str} < \text{Vec}$)을 기반으로 수행되며, 벡터 내부에 또 다른 벡터가 포함된 경우 재귀적 비교를 통해 사전식(Lexicographical) 정렬을 수행한다. 또한 `(sort v 'desc)`와 같이 모드를 지정하면 내림차순 정렬도 지원한다.

제3장 프로그래밍 언어 구현

3.1 구현 방향 및 시스템 개요

3.1.1 인터프리터 방식의 채택

본 프로젝트의 언어 처리기는 **인터프리터(Interpreter)** 방식으로 구현되었다. 컴파일러 방식은 소스 코드를 기계어로 번역하여 실행 파일을 생성하는 반면, 인터프리터 방식은 코드를 한 줄씩 읽어 즉시 실행 결과를 반환한다. 본 언어가 LISP 기반의 대화형 셸(REPL: Read-Eval-Print Loop) 환경을 지향하고 있으며, Python과 유사한 동적 타이핑(Dynamic Typing) 및 이기종 벡터(Heterogeneous Vector)를 지원하기 위해서는 런타임에 타입을 검사하고 메모리를 관리하는 인터프리터 방식이 적합하다고 판단하였다.

3.1.2 시스템 구성도 (System Architecture)

전체 시스템은 어휘 분석(Lexical Analysis), 구문 분석(Syntax Analysis), 평가(Evaluation), 그리고 출력(Printing)의 4단계 파이프라인으로 구성된다. 데이터의 흐름(Data Flow)은 다음과 같다.

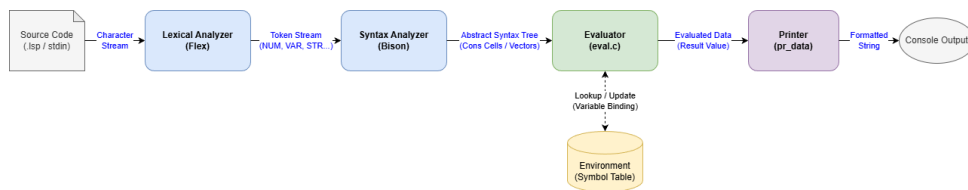


그림 3.1: LISP 인터프리터의 시스템 구성도 및 데이터 흐름

3.2 구문 분석 및 AST 생성

3.2.1 어휘 분석과 토큰 처리 (Lexer)

어휘 분석 단계에서는 입력 스트림을 스캔하여 토큰을 생성한다. 특히 본 언어는 문자열 내부의 이스케이프 시퀀스 처리를 지원해야 하므로, Lex 단계에서 이를 처리하도록 구현하였다.

다음은 문자열 리터럴을 처리하는 `token.l`의 핵심 코드이다.

코드 3.1: 문자열 토큰 처리 및 이스케이프 시퀀스 변환 (`token.l`)

```

1  /* String pattern that allows escaped quotes */
2  str      \"(\\.|[^\"])*\"
3
4  %%
5  {str} {
6      /* Allocate memory excluding quotes */
7      char *s = (char*)malloc(yleng - 1);
8      /* ... Parsing loop handling \n, \", etc. ... */
9      yylval.sval = s;
10     return STRING;
11 }

```

3.2.2 일반화 리스트를 이용한 AST 구조

LISP의 가장 큰 특징은 "코드와 데이터의 형태가 같다(Homoiconicity)"는 점이다. 따라서 별도의 복잡한 AST 구조체를 정의하는 대신, LISP의 기본 자료구조인 **Cons Cell(일반화 리스트)**을 그대로 AST로 사용한다.

예를 들어, 소스 코드 `(+ 1 2)`는 파싱 단계에서 다음과 같은 리스트 구조로 메모리에 생성된다.

- Root $\rightarrow$ Cons(Symbol("+"), Cons(Int(1), Cons(Int(2), NIL)))

단, 벡터 구문 `[ 1 2 ]`의 경우, 리스트와 달리 파싱 시점에 즉시 벡터 객체로 변환되어 AST의 단말 노드(Atom)처럼 취급된다. 이를 위해 `exprs_to_vec` 함수가 사용된다.

코드 3.2: 벡터 생성을 위한 파서 보조 함수 (`ast.y`)

```

1  static Data *exprs_to_vec(Data *list) {
2      Data *vec = mkvec();
3      if (!vec) return mknil();
4
5      Data *cursor = list;
6      while (!is_nil(cursor)) {
7          Data *val = fst(cursor);
8          vec_push(vec, val);
9          cursor = snd(cursor);

```

```

10     }
11     return vec;
12 }

```

3.3 타입 시스템 및 메모리 구조

3.3.1 동적 타이핑과 Tagged Union

본 언어는 이기종 벡터(Heterogeneous Vector)를 지원하기 위해 런타임에 데이터 타입을 식별할 수 있어야 한다. 이를 위해 모든 데이터는 Tagged Union 구조체인 Data로 캡슐화된다. Tag 열거형을 통해 현재 데이터가 정수인지, 문자열인지, 혹은 벡터인지 식별한다.

코드 3.3: 데이터 타입 정의 및 Tagged Union 구조 (data.h)

```

1  typedef enum {
2      TNAT, TCHR, TSTR, TSYM, TNIL, TDUO, TOPR, TVEC
3  } Tag;
4
5  typedef struct _Data {
6      Tag tag;
7      union {
8          Nat nv;      // Unsigned Integer
9          Chr cv;      // Character
10         Str sv;      // String / Symbol
11         Vec vv;      // Vector (Dynamic Array)
12         Duo pv;      // Pair (Cons Cell)
13         Opr fv;      // Function Pointer
14     };
15 } Data;

```

3.3.2 타입 검사 (Type Checking)

정적 컴파일 언어와 달리, 본 인터프리터는 의미 분석(Semantic Analysis) 단계가 평가(Evaluation) 단계와 통합되어 있다. 연산자가 실행될 때 피연산자의 tag를 검사하여 타입 일치 여부를 확인한다. 타입이 호환되지 않는 경우(예: 벡터와 정수의 덧셈 시도) NIL을 반환하거나 에러를 처리한다.

3.4 평가기(Evaluator) 및 연산자 구현

3.4.1 다형성 연산자 구현 (Polymorphism)

본 프로젝트의 핵심 기능인 다형성 연산은 `bop.c` 내의 함수들에서 구현된다. 예를 들어, 덧셈 연산자 `+`는 입력된 인자들의 타입을 검사하여 **벡터 병합**, **문자열 연결**, **정수 덧셈** 중 적절한 동작을 수행한다.

코드 3.4: 다형성 덧셈 연산자 구현 (`bop.c`)

```

1 Data *bop_add(Data *args) {
2     /* ... (Type Detection Logic) ... */
3
4     /* Case 1: Vector Concatenation */
5     if (is_vec_mode) {
6         Data *new_vec = mkvec();
7         /* ... Iterate and push elements ... */
8         return new_vec;
9     }
10
11    /* Case 2: Numeric Addition */
12    if (!is_str_mode) {
13        /* ... Sum integers or chars ... */
14        return mknat(sum);
15    }
16
17    /* Case 3: String Concatenation */
18    /* ... Allocate buffer and strcat ... */
19    return mkstr(buf);
20 }

```

3.4.2 벡터 연산 및 정렬 (Vector Operations)

벡터는 C언어의 동적 할당 배열을 기반으로 구현되었다. `vop.c` 모듈은 벡터의 삽입, 삭제, 슬라이싱 등을 수행하며, 특히 `sort` 함수는 재귀적 비교 로직을 통해 중첩된 벡터까지 정렬할 수 있다.

코드 3.5: 재귀적 비교를 이용한 정렬 로직 (`vop.c`)

```

1 static int compare_data(Data *d1, Data *d2) {

```

```
2      /* 1. Check type hierarchy */
3      if (d1→tag != d2→tag) return d1→tag - d2→tag;
4
5      /* 2. Recursive comparison for Vectors */
6      if (d1→tag == TVEC) {
7          /* ... Lexicographical comparison ... */
8          int cmp = compare_data(e1, e2); // Recursive call
9          if (cmp != 0) return cmp;
10     }
11     /* ... Handle other types ... */
12 }
```

제4장 테스트 코드와 테스트 결과

본 장에서는 앞서 설계 및 구현한 LISP 인터프리터의 기능 검증을 위해 수행한 5가지 주요 테스트 시나리오와 그 결과를 기술한다. 테스트는 리눅스 셸 환경에서 미리 작성된 .lsp 스크립트 파일을 인터프리터의 표준 입력(Standard Input)으로 리다이렉션하여 수행하였다.

4.1 테스트 환경 및 출력 형식

테스트 스크립트 파일(*.lsp)을 일괄 실행할 때, 입력된 코드가 화면에 보이지 않는 문제를 해결하기 위해 인터프리터 내부적으로 입력 구문을 에코(Echo) 출력하도록 구현하였다. 결과 화면의 각 라인은 다음과 같은 의미를 가진다.

- `=== Title ===`: 해당 테스트 시나리오의 대주제
- `-- Subtitle --`: 테스트하는 세부 기능 그룹
- `> (func ...)`: **사용자 입력 코드**. 실제 인터프리터에 입력된 구문이다.
- `Result: >` 기호가 없는 라인은 해당 구문의 평가(Evaluation) 결과값이다.
- `Note: ...`: 테스트 이해를 돕기 위한 주석 또는 부가 설명이다.

4.2 기초 연산 및 변수 처리 (Basic Operations)

첫 번째 테스트는 인터프리터의 가장 기초적인 기능인 변수 할당, 정수 산술 연산, 그리고 LISP의 기본 자료구조인 Cons Cell 처리를 검증한다. 특히 본 프로젝트에서 추가된 Character 타입과 정수 간의 아스키(ASCII) 연산이 정상적으로 수행되는지 확인한다.

```

$ ./main.exe < test_01_basics_char.lisp
=== 1. Variables & Character Arithmetic ===
--- Variables ---
> (= a 10)
> (= b 20)
> a
10
> b
20
> (+ a b)
30
> (* a b)
200
--- Character Math (ASCII) ---
Note: 'a' is 97, 'b' is 98
> (= ch 'a')
1. Char + Int ('a' + 3 -> 100)
> (+ ch 3)
100
2. Char + Char ('a' + 'b' -> 195)
> (+ 'a' 'b')
195
3. Char * Int ('a' * 2 -> 194)
> (* 'a' 2)
194
--- Cons Cells (LISP List) ---
> (= list (cons 1 (cons 2 (cons 3 ())))))
> (car list)
1
> (cdr list)
(2 . (3 . ()))
> (car (cdr list))
2

```

그림 4.1: 기초 연산, 문자(Char) 산술, 리스트 처리 테스트 결과

그림 4.1을 보면 정수형 변수 a, b의 사칙연산이 정확히 수행됨을 알 수 있다. 또한 문자 'a'(97)에 정수 3을 더해 100을 반환하거나, 문자끼리 더하는 연산이 정상적으로 처리되었다. 하단의 cons 중첩 호출 결과 역시 (1 . (2 . (3 . ()))) 형태의 올바른 리스트 구조를 생성하고 있다.

4.3 문자열 다형성 연산 (Polymorphic String Ops)

두 번째 테스트는 Python과 유사하게 설계된 '다형성 연산자'의 동작을 검증한다. 문자열에 대한 덧셈(+)과 곱셈(*) 연산자가 피연산자의 타입에 따라 어떻게 동작하는지 확인한다.

```

$ ./main.exe < test_02_poly_str.lsp
=== 2. Polymorphic String Operations ===
--- String Concatenation ---
> (= s1 "Hello")
> (= s2 " World")
> (+ s1 s2)
Hello World
--- Mixed Type Concatenation ---
1. String + Integer
> (+ "Value: " 100)
Value: 100
2. String + Character (Append Char)
> (+ "Grade: " 'A')
Grade: A
3. Complex Mix
> (+ "Result: " 10 " points (" 'P' ")")
Result: 10 points (P)
--- String Repetition ---
> (* "Go! " 3)
Go! Go! Go!

```

그림 4.2: 문자열 결합 및 반복 연산 테스트 결과

그림 4.2의 결과에서 (+ s1 s2)는 두 문자열을 연결하였고, (+ "Value: " 100)과 같이 문자열과 정수, 혹은 문자가 혼합된 경우에도 자동으로 형 변환되어 연결됨을 확인할 수 있다. 또한 (* "Go! " 3) 명령을 통해 문자열 반복 기능이 정상 동작함을 입증하였다.

4.4 벡터 생성 및 조회 (Vector Read Operations)

세 번째 테스트는 본 언어의 핵심 확장 기능인 벡터(Vector)의 생성과 조회 기능을 다룬다. 벡터 리터럴 [...]을 통한 생성과 길이 확인(len), 인덱싱(idx), 슬라이싱(slice), 선형 탐색(find) 기능을 테스트한다.


```

$ ./main.exe < test_03_vector_read.lsp
=== 3. Vector Read Operations ===
--- Creation & Length ---
> (= v [10 20 30 40 50])
> v
[10 20 30 40 50]
> (len v)
5
--- Indexing (idx) ---
> (idx v 0)
10
> (idx v 4)
50
Error Case (Out of Bound) -> NIL:
> (idx v 999)
()
--- Slicing (slice) ---
Slice [1:4]:
> (slice v 1 4)
[20 30 40]
--- Linear Search (find) ---
Find 30:
> (find v 30)
2
Find 99 (Missing) -> NIL:
> (find v 99)
()

```

그림 4.3: 벡터 생성, 인덱싱, 슬라이싱 및 탐색 테스트 결과

그림 4.3과 같이 벡터는 생성 즉시 평가되어 출력된다. `idx` 함수는 유효한 인덱스에 대해서는 값을 반환하고, 범위를 벗어난 경우(999)에는 NIL을 반환하여 안전성을 보장한다. `slice` 함수는 원본 벡터의 부분 복사본을 정상적으로 생성하였으며, `find` 함수는 요소의 위치를 정확히 반환하였다.

4.5 동적 벡터 수정 (Vector Modification)

네 번째 테스트는 벡터의 동적 메모리 관리 능력을 검증한다. 벡터를 스택(Stack)처럼 사용하는 `push/pop` 연산과 임의 위치에 대한 삽입(`ins`), 삭제(`del`), 수정(`set`) 기능을 확인한다.

```

$ ./main.exe < test_04_vector_write.lsp
=== 4. Vector Modification (Dynamic Array) ===
> (= st [1 2])
--- Stack: Push & Pop ---
> (push st 3)
[1 2 3]
> (push st 4)
[1 2 3 4]
> st
[1 2 3 4]
Pop returns:
> (pop st)
4
Stack after pop:
> st
[1 2 3]
--- Random Access Modification ---
> (= v ["a" "b" "c"])
> (set v 1 "changed")
[a changed c]
> v
[a changed c]
--- Insert & Delete ---
> (ins v 1 "inserted")
[a inserted changed c]
> v
[a inserted changed c]
> (del v 0)
a
> v
[inserted changed c]

```

그림 4.4: 벡터의 동적 삽입, 삭제 및 스택 연산 테스트 결과

그림 4.4에서 초기 크기가 2였던 벡터에 push 연산을 수행함에 따라 크기가 동적으로 늘어나고 데이터가 추가되는 것을 확인할 수 있다. 또한 ins와 del 연산을 통해 배열 요소들이 자동으로 시프트(Shift)되며 데이터 무결성이 유지됨을 보여준다.

4.6 고급 기능: 벡터 산술 및 재귀 정렬 (Advanced Features)

마지막 테스트는 벡터 간의 연산과 재귀적 정렬 알고리즘을 검증한다. 벡터끼리의 덧셈(병합), 벡터와 정수의 곱셈(반복 확장), 그리고 이기종 데이터가 섞인 중첩 벡터의 정렬 결과를 확인한다.

```

$ ./main.exe < test_05_adv_sort.lsp
=== 5. Advanced Sort & Vector Math ===
--- Vector Arithmetic ---
> (= v1 [1 2])
> (= v2 [3 4])
1. Vector + Vector (Concat):
> (+ v1 v2)
[1 2 3 4]
2. Vector * Int (Repeat):
> (* v1 3)
[1 2 1 2 1 2]
--- Sorting (Basic) ---
> (= nums [5 2 8 1 9])
> (sort nums asc)
[1 2 5 8 9]
> (sort nums desc)
[9 8 5 2 1]
--- Deep Recursive Sort (Nested Vectors) ---
Before: [[3 1] [1 2] [1] [1 2 3]]
> (= nested [[3 1] [1 2] [1] [1 2 3]])
After Sort (Python-style comparison):
Expected Order: [1] < [1 2] < [1 2 3] < [3 1]
> (sort nested)
[[1] [1 2] [1 2 3] [3 1]]

```

그림 4.5: 벡터 산술 연산 및 중첩 벡터 재귀 정렬 테스트 결과

그림 4.5의 상단부는 두 벡터 [1 2]와 [3 4]가 덧셈 연산을 통해 [1 2 3 4]로 병합되는 과정을 보여준다. 하단부의 정렬 테스트에서는 [[3 1] [1 2] [1] [1 2 3]]과 같은 복잡한 중첩 벡터가 입력되었을 때, 내부 요소의 길이와 값을 재귀적으로 비교하여 올바른 순서로 정렬됨을 확인할 수 있다. 이는 본 인터프리터가 단순한 1차원 배열을 넘어 복합 자료구조를 완벽히 제어할 수 있음을 시사한다.

제5장 고찰 (Reflections)

본 프로젝트는 기존 LISP 인터프리터를 확장하는 과정에서 다양한 설계 선택과 기술적 결정을 필요로 하였다. 특히 타입 확장, 벡터 자료구조 도입, 다형성 연산 구현은 단순한 기능 추가를 넘어 언어의 근본적 의미 체계와 실행 모델을 다시 정의해야 하는 작업이었다. 본 장에서는 구현 과정에서 직면했던 주요 문제들, 그에 대한 해결 전략, 그리고 대안적 접근에 대한 고찰을 중심으로 논의한다.

5.1 타입 확장의 필요성과 설계적 선택

초기 인터프리터는 정수 타입만을 지원했기 때문에 표현 범위가 매우 제한적이었다. 타입 확장은 단순 기능 추가라기보다, 언어가 어떤 데이터 모델을 기반으로 작동해야 하는지를 새롭게 정립하는 과정이었다. 이를 위해 문자(Character)와 문자열(String)을 도입하는 것은 자연스러운 선택이었지만, 내부 표현 방식을 어떻게 확장할 것인지에 대해 여러 가능성을 검토할 필요가 있었다.

특히 벡터(Vector) 자료구조를 설계할 때 가장 큰 고민은 벡터가 단일 타입만 허용하는 정적 컨테이너가 되어야 하는가, 혹은 서로 다른 타입의 값들을 동시에 담을 수 있는 이기종 컨테이너가 되어야 하는가였다. 이는 C++의 `std::vector<int>`처럼 타입을 엄격히 고정하는 방식과, Python의 리스트나 JavaScript의 배열처럼 어떤 타입도 섞어서 저장할 수 있는 방식 중 어떤 철학을 따를지 결정하는 문제였다.

단일 타입 벡터를 선택할 경우 타입 안정성이 보장되고 연산자 의미도 단순해지지만, 본 프로젝트의 목표인 “간단하고 유연한 스크립트 언어”라는 방향성과는 다소 거리가 있었다. 반면 이기종 벡터는 동적 언어의 표현력을 크게 확장할 수 있으나, 내부 타입 관리와 연산 정의가 복잡해지는 단점이 있었다.

최종적으로는 이기종 벡터를 허용하는 방향을 선택하였다. 이는 기존 실습 단계에서 이미 Tagged Union 기반 데이터 구조를 사용하고 있었기 때문이 아니라, 정수-문자-문자열-벡터가 자유롭게 혼합될 수 있는 표현력이 본 프로젝트의 목표에 더 부합한다고 판단했기 때문이다. Tagged Union은 이러한 결정을 자연스럽게 뒷받침할 수 있는 구조였고, 타입 검사와

연산 분기 로직도 명확하게 구현할 수 있었다.

결과적으로 이 선택은 다형성 연산, 벡터 정렬, 중첩 벡터 처리 등 이후 확장 기능을 구현하는 데 중요한 기반이 되었으며, 단일 타입 벡터보다 더 큰 유연성을 제공하는 방향으로 언어의 성격을 규정하게 되었다.

5.2 다형성 연산 구현 과정에서의 고민

Python의 문자열 결합이나 반복 연산처럼, 타입에 따라 동일 연산자가 서로 다른 의미를 가지도록 만드는 것은 직관적인 사용자 경험을 제공하지만, 구현자 입장에서는 타입 간 우선순위와 모호성 문제를 해결해야 한다.

예를 들어 문자열과 정수를 더하는 경우를 어떻게 다룰 것인지 고민이 필요했다. 처음에는 엄격한 타입 규칙을 적용해 오류를 반환하는 방식도 고려했지만, 사용성과 언어 철학을 고려해 문자열로 자동 변환하여 결합하도록 결정하였다. 이 과정에서 "어디까지 자동 변환을 허용할 것인가"를 세밀하게 조정해야 했고, 실제로 여러 테스트를 수행하며 일관성을 유지하도록 규칙을 다듬었다.

또한 벡터 덧셈을 벡터 병합으로 정의한 것도 구현 중 중대한 결정이었다. 기존 LISP의 리스트 연산과 충돌하지 않으면서도 직관적인 기능을 제공하려면, +를 '연결'의 의미로 확장하는 것이 가장 자연스럽다고 판단하였다.

5.3 벡터 자료구조 설계와 대안 비교

벡터(Vector)는 이번 프로젝트에서 가장 많은 설계적 고민을 요구한 기능이었다. 특히 다음과 같은 문제를 해결해야 했다.

- 동적 크기 조절을 어떤 방식으로 구현할 것인가?
- 중첩 벡터를 비교할 때 어떤 기준을 사용할 것인가?
- 리스트와 벡터의 역할을 어떻게 명확히 구분할 것인가?

초기에는 단순한 정적 배열을 기반으로 구현하는 방식도 고려했지만, 이 경우 삽입·삭제가 매우 비효율적이 되며, 크기 변경 시 재할당 로직을 직접 구현해야 하는 문제가 있었다. 결국 C 표준 동적 메모리 관리(malloc, realloc)를 활용해 capacity-size 모델을 적용하는 것이 가장 설계적으로 깔끔한 해법이었다.

중첩 벡터 정렬의 경우, 비교 함수가 재귀적으로 호출되도록 설계하였으며, 이는 결과적으로 Python의 리스트 정렬 방식과 유사한 동작을 갖도록 만들어주었다. 다만 이 접근은 구현

난이도를 높였고, 타입 계층(Tag Priority)을 세밀하게 정의해야만 안정적 동작이 가능했다. 이 과정에서 여러 차례 테스트와 디버깅을 반복하며 비교 규칙을 정교하게 다듬을 필요가 있었다.

5.4 AST 구성 방식 선택에 대한 고찰

LISP의 핵심 특성인 Homoiconicity 덕분에 별도의 AST 구조체를 설계하는 대신 cons 셀을 AST로 재활용할 수 있었다. 이는 구현 비용을 크게 줄여준 선택이지만, 한편으로는 LISP의 문법적 제약에서 자유롭지 않기 때문에 확장성 측면에서는 한계가 있다.

예를 들어 복잡한 구문을 지원하는 일반적인 언어(예: C, Java)의 경우에는 별도의 AST 노드 타입 정의가 필수적이다. 본 프로젝트에서는 cons 기반 AST가 가장 적합했지만, 다양한 실험적 확장을 시도하는 과정에서 "전통적인 AST 구조체를 병행했다면 더 명확한 표현이 가능했을까?"라는 고민도 남았다. 이는 추후 인터프리터를 발전시킬 때 고려할 만한 지점이다.

5.5 인터프리터 방식 선택의 타당성

프로젝트 초기에는 TVM 코드 생성을 통한 간단한 컴파일러 방식도 후보로 검토했으나, 확장 타입과 벡터 기능을 고려하면 인터프리터 방식이 훨씬 자연스러웠다. 특히 동적 타이핑, 런타임 타입 검사, 이기종 컨테이너는 컴파일러보다 인터프리터에서 구현하는 것이 수월하다.

재귀적 정렬이나 벡터 연산처럼 런타임에 구조가 확정되는 기능은 컴파일 타임 정보만으로는 최적화가 어렵기 때문에, 이번 프로젝트에서 인터프리터 방식을 선택한 것은 충분히 타당했다고 판단한다.

5.6 전체 구현 과정에서 얻은 교훈

본 프로젝트에서 가장 크게 느낀 점은, 작은 언어라도 “표현력 높은 언어를 만드는 것은 단순 기능 구현의 집합이 아니라, 언어 철학과 사용 경험을 함께 설계하는 작업”이라는 것이다. 타입 체계, 연산 의미, 자료구조 설계는 서로 깊게 연관되어 있으며, 한 부분의 선택이 전체 언어의 일관성과 안정성에 직접적인 영향을 미친다.

또한, 직접 인터프리터를 설계하고 구현하는 경험은 실제 프로그래밍 언어가 어떤 구조 위에서 동작하는지 근본적인 감각을 익히는 데 큰 도움이 된다. 단순히 이론을 학습하는 것이 아니라, 설계-구현-디버깅의 반복을 통해 언어 내부 구조를 몸으로 이해할 수 있었다는 점에서 값진 학습 경험이었다.

제6장 결론

본 프로젝트는 기존에 구현되어 있던 간단한 LISP 인터프리터를 기반으로, 보다 다양한 데이터 타입과 실용적인 연산을 지원하도록 기능을 확장하는 것을 목표로 진행되었다. 핵심 확장 요소는 문자(Character)와 문자열(String) 타입의 도입, 연산자 오버로딩을 통한 일관된 산술·문자열 처리, 그리고 새로운 벡터(Vector) 자료구조의 추가이다. 벡터는 배열 기반의 구조로 설계하여 CRUD, 탐색, 슬라이싱, 정렬 등 스크립트 언어에서 자주 사용되는 기능을 실험적으로 구현할 수 있었다.

프로젝트 과정에서 어휘 분석, 구문 분석, 평가기의 동작 방식 등 언어 처리기의 주요 구성 요소를 다시 확인할 수 있었으며, 작은 기능을 추가하더라도 전체 파이프라인의 의미론적 일관성을 유지하는 것이 중요함을 경험했다. 특히 벡터 연산을 설계하고 구현하는 과정은 자료구조 선택, 타입 처리, 의미 정의가 서로 연결되어 있다는 점을 명확하게 보여주었다.

최종 결과물은 교육용 인터프리터로서 필요한 기능들을 안정적으로 수행하며, 단순한 언어라도 구조를 갖추어 확장해 나가는 과정이 어떤 식으로 이루어지는지 체감할 수 있는 수준의 완성도를 갖추었다고 판단한다. 추후 시간이 있다면 map, filter 같은 고차 연산자나 보다 일관된 예외 처리, 간단한 메모리 관리 기능 등을 추가하여 확장 실험을 이어갈 수 있을 것이다.

이번 프로젝트는 언어 구현의 전체 흐름을 실제 코드로 경험해 볼 수 있었던 작업이며, 해석기 내부 구조와 자료구조 설계 간의 관계를 이해하는 데 도움이 되었다. 이를 통해 이후 더 복잡한 언어나 시스템을 다루게 될 때 기본적인 발판 역할을 할 것으로 기대한다.

참고문헌

- [1] G. Woo, *Compiler*, School of Computer Science and Engineering, Pusan National University, 2025.
- [2] A. Aho, M. Lam, R. Sethi, J. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Addison-Wesley, 2006.
- [3] H. Abelson and G. J. Sussman, *Structure and Interpretation of Computer Programs*, 2nd ed., MIT Press, 1996.
- [4] R. Hickey, “The Clojure Programming Language,” <https://clojure.org/>, Checked on Dec. 20, 2025.
- [5] Python Software Foundation, “The Python Standard Library: Sequence Types,” <https://docs.python.org/3/library/stdtypes.html>, Checked on Dec. 20, 2025.